# HertsBike

A Simulated SQLite Database for a Local Bike Share Scheme

| | |
|---|---|
| Student | **Rajesh** |
| Student ID | **25051282** |
| Module | **Data Science** |
| Assignment | **SQL Database Project** |

## 1. Purpose of database

This database represents a fictional bike share scheme called 'HertsBike'. There are 40 docking stations across 8 towns in Hertfordshire. It was chosen because it contains all 4 levels of measurement. There are nominal variables (towns and bike types), ordinal (membership tiers and ratings), interval (weather has a scale of temperatures with an arbitrary zero) and ratio (trips have lengths, distances and prices). Bikes also have a maintenance history. This could be used by an operator to see which stations have a shortage of free docks at 8am, if rain affects ratings or which bikes have the highest maintenance costs.

## 2. Generating the data

The database was created using a Python notebook with the numpy, pandas and sqlite3 libraries. All data was generated using a fixed seed (2026) so you can recreate the database exactly. No data was downloaded from external sources. All names were created from lists I created. Emails were created using the individual member's name. Phone numbers are in the UK format (07xxx). Trips were created using a series of rules (see Table 1).

| Column | How it was simulated |
|---|---|
| avg_temp_c, rainfall_mm | avg_temp_c follows a yearly sine wave and carries forward the daily anomaly so there are periods of warm and cold weather. rainfall_mm is 0 on dry days and gamma distributed on rainy days. |
| Trip dates and times | Stations are chosen based on popularity. Dates are weighted according to the weather of that town (there are more rides on warmer and drier days). Between 1,446 and 2,981 trips were made each month. Trips started at peak times around 08:15 and 17:30 on weekdays. |
| Members and bikes | Members and bikes have log-normal weights for activity, with higher tier members having higher weights. Only members who have signed up and bikes that have already been purchased can be used. |
| Distance, duration and price | Distance is the actual distance between stations multiplied by 1.3. Duration is distance divided by the speed of the bike type. Price depends on the tier of the member. Casual members pay £1.00 to unlock and 15p per minute. Standard and Premium members have 30 and 60 minutes free. |
| Rating | Ratings are skewed to 4 and 5. On days where there is more than 5mm of rain, ratings are lower. |

*Table 1: Rules used to create the simulated database.*

There was intentional missing data and duplicate data. The notebook contains all rates in one dictionary and applies them in explicit code. All duplicate members are recorded. They still ride so they cause issues when cleaning.

| Where | Rate | Reason | Result |
|---|---|---|---|
| members.email | 6% missing | Email wasn't required to sign up | 82 rows (5.6%) |
| members.phone | 12% missing | Optional | 180 rows (12.4%) |
| members.birth_year | 5% missing | Optional | 70 rows (4.8%) |
| trips.rating | 40% missing | Often ignored | 10,282 rows (40.5%) |
| trips.distance_km | 3% missing | GPS not working | 753 rows (3.0%) |
| weather.rainfall_mm | 2% missing | Rain gauge not working | 66 rows (2.3%) |

| Where | Rate | Reason | Result |
|---|---|---|---|
| maintenance.cost_gbp | 4% missing | Invoice not matched | 129 rows (3.8%) |
| Duplicate members | 4% re-registered | Forgot login details | 56 additional rows |
| Duplicate trips | 1.5% re-uploaded | Synchronised twice | 375 additional rows |

*Table 2: Missing and duplicate data, with the rates achieved. Duplicate members have their own member_id and sign up later. Duplicate trips have their own trip_id.*

## 3. Database structure

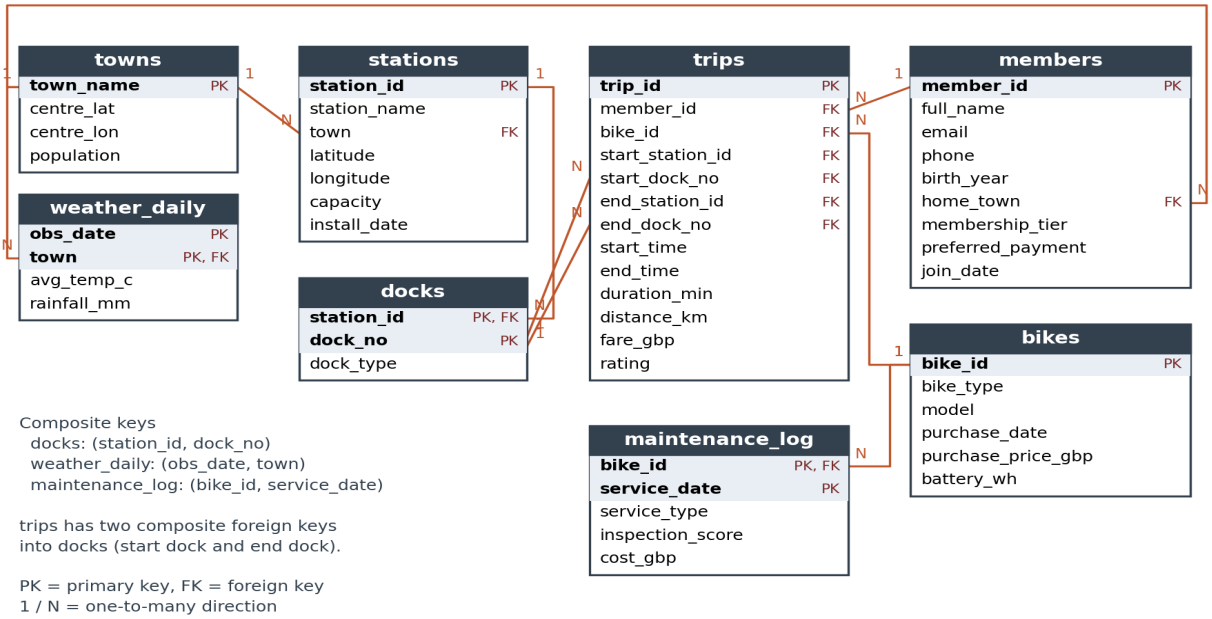



*Figure 1: Database structure. Orange lines show foreign keys.*

| Table | Rows | Primary key | Foreign key(s) |
|---|---|---|---|
| towns | 8 | town_name | |
| stations | 40 | station_id | town |
| docks | 836 | (station_id, dock_no) | station_id |
| bikes | 320 | bike_id | |
| members | 1,456 | member_id | home_town |
| weather_daily | 2,920 | (obs_date, town) | town |
| maintenance_log | 3,357 | (bike_id, service_date) | bike_id |
| trips | 25,375 | trip_id | member_id, bike_id, (start_station_id, start_dock_no), (end_station_id, end_dock_no) |

*Table 3: Tables in the database and their relationships.*

| Scale | Columns |
|---|---|
| Nominal | town, bike_type, model, dock_type, service_type, preferred_payment, full_name, email |
| Ordinal | membership_tier (Casual < Standard < Premium), rating, inspection_score |
| Interval | avg_temp_c, birth_year, dates/times |
| Ratio | duration_min, distance_km, fare_gbp, rainfall_mm, cost_gbp, purchase_price_gbp, capacity, battery_wh |

*Table 4: Scales for each column.*

## 4. Reasoning Behind Tables and Constraints

Each table contains information that changes independently. To avoid the same information (e.g. station names, towns and bike models) being repeated 25,000 times, only keys are stored in the trips table. The towns table is small so it acts as a reference table for stations, members and weather. Weather data is recorded for each town each day so it is stored separately. The maintenance data is stored separately as each bike has multiple services (3,357 rows for 320 bikes).

3 tables have composite primary keys. Dock numbers don't make sense without knowing the station, the weather data has a date and town, and bikes can't be serviced twice in a day. There are 2 composite foreign keys in the trips table, meaning a trip cannot begin or end at a non-existent dock. After completing the database, SQLite checked the foreign keys and found no issues.

All attributes have constraints. Values are within ranges (ratings between 1-5, capacity between 8-40, duration between 1-1440 minutes, no negative prices). IN statements limit categories. End time must be after start time. Battery only for electric bikes. Only columns that would have real gaps can be NULL. trip_id is a surrogate key. Email isn't UNIQUE so there can be duplicates, as there would be in a system that never de-duplicated sign-ups.

## 5. Ethics and Privacy

There are no real people in this database. Names are made from combinations of my own lists. A generated name and email could match a real person by coincidence, so it shouldn't be used to contact anyone. If real data was used this would be sensitive. Names, emails, phone numbers and birth years are personal data under UK GDPR. Trips have a time so you can see patterns. If someone leaves the same station at 08:15 every weekday they have effectively given away where they live and work. In a real scheme only the necessary information should be collected (birth year not date of birth, contact details are optional). Identity should be kept in a separate table with access restrictions. Pseudonymous IDs should be used in trips. A retention period should be set and locations blurred before sharing. Small groups can be an issue. If there is only one Premium member aged 70 in Harpenden they can be identified from the small amount of attributes.

In real life missing ratings and GPS drops are rarely random. Duplicate accounts will make the member count higher. The data isn't clean so can be misleading. My lists of names are short and not representative so nothing here is evidence about real communities. The data was created to practise SQL and data preparation.

## 6. Code

The Python code used to generate the database is available on GitHub:
https://github.com/RajeshMandala1/HertsBike_generator